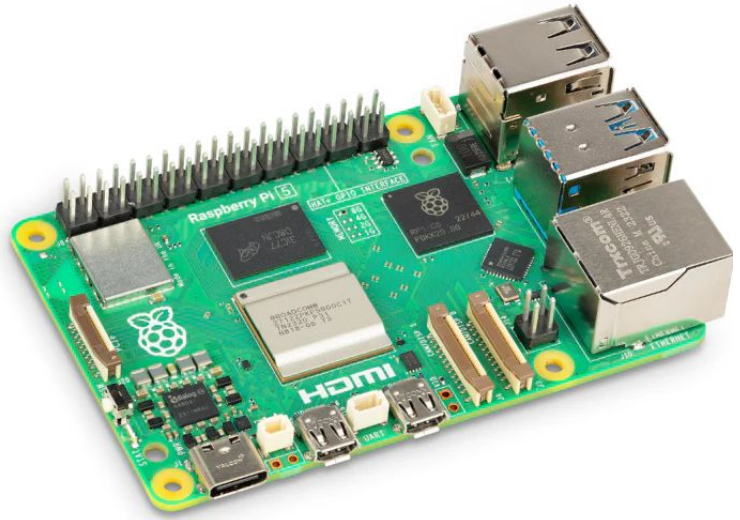
Building a mini - HPC cluster

Step 1 - Flashing the Operating System to SD Card

- Raspberry Pi does **not come with an OS**
- We must install (flash) an operating system on SD card
- This SD card becomes the **boot disk** of each node

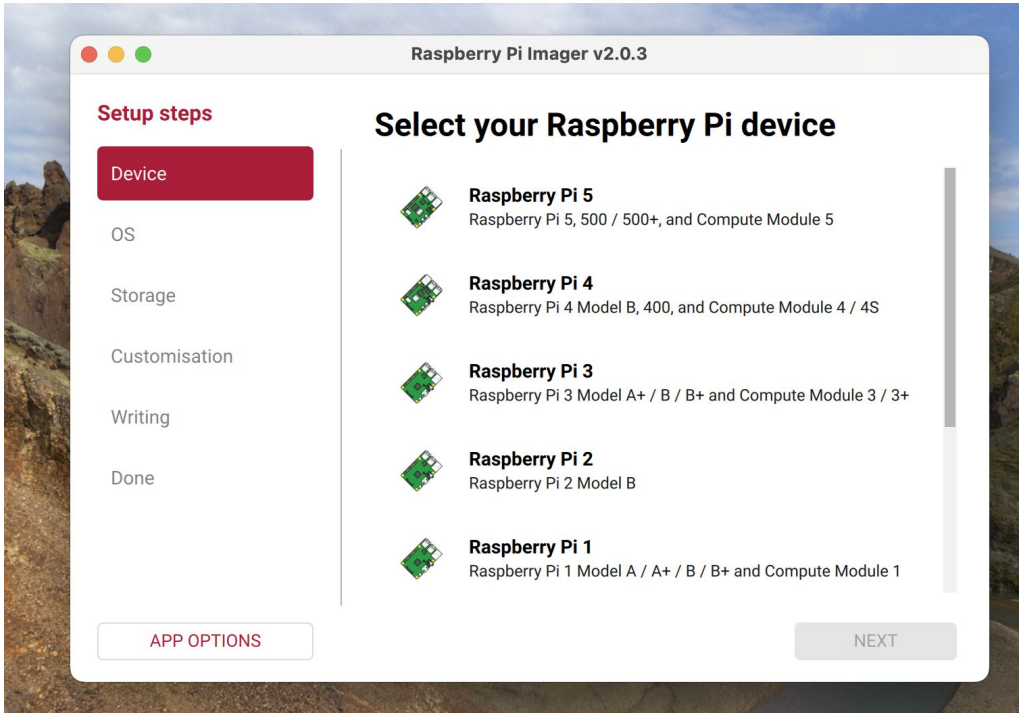


Step 1 - Flashing the Operating System to SD Card

- **Download Raspberry Pi Imager**
- Website: https://www.raspberrypi.com/software/
- Works on:
  - Windows
  - macOS
  - Linux

Step 1 - Flashing the Operating System to SD Card

- Insert SD card into laptop
- Open Raspberry Pi Imager
- Select OS
- Select SD card
- Enable **SSH**
- Set **hostname**
- Set **username & password**
- Click **Write**



Naming Convention for Cluster

Master Node (Head/Login Node)

- **Hostname:** master
- **Username:** hpc
- **Password:** pi
- **IP Address:** 172.28.8.3

How to use ssh?

ssh hpc@172.28.8.3
ssh hpc@ip_address_pi

Hostname	IP Address
Node 1	172.28.8.4
Node 2	172.28.8.5
Node 3	172.28.8.6
Node 4	172.28.8.7
Node 5	172.28.8.8
Node 6	172.28.8.9

Step 2 - Powering On the Raspberry Pi

- Insert **SD card** into Raspberry Pi
- Connect **HDMI cable → Monitor**
- Connect **USB Keyboard**
- Connect **Ethernet cable**
- Plug **Power Adapter**
- **Set WLAN country ***

Step 3 - Boot Order & SSH Enable

- Set WLAN country (Localisation options)
- Enable SSH (Interface options)

Sudo raspi-config

Step 4 - Network Configuration

Instructions

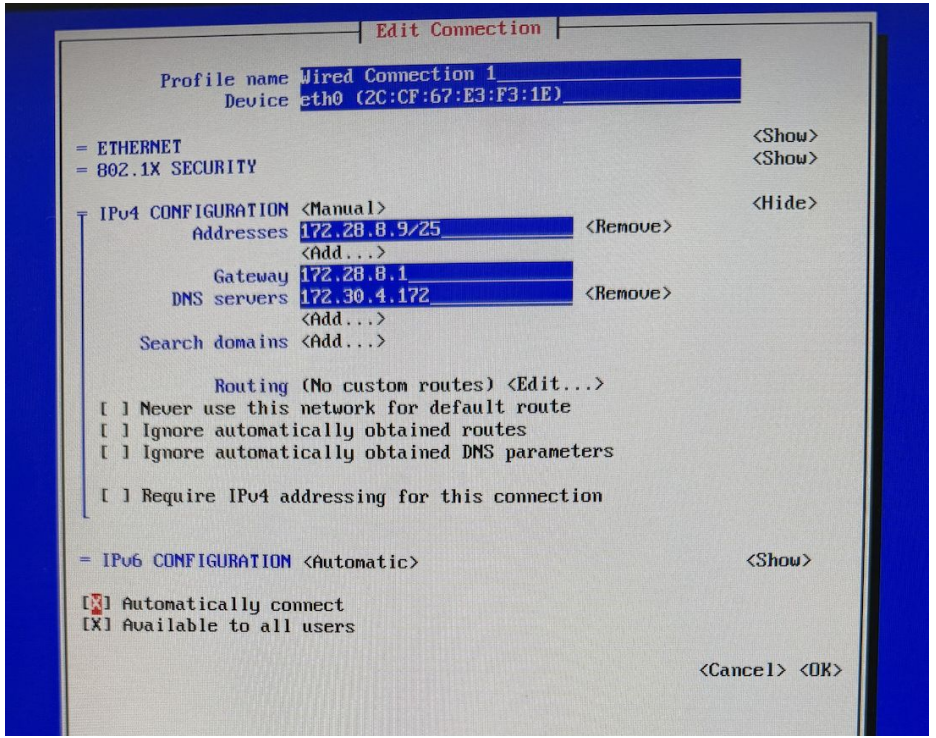
- Delete default network connection
- Create new wired connection
- Assign static IP address
- Connect to switch
- Bring connection up

Commands:

- ***nmtui*** (Delete the previous connection)
- ***nmcli connection add type ethernet ifname eth0 con-name “Wired connection 1”***
  (Creates a new wired Ethernet network profile named “Wired connection 1” for interface eth0.)

Step 4 - Network Configuration

- Edit a connection and give IP address
- Up the connection
  nmcli connection up "Wired connection 1"



Check if you are connected to the Internet.

Step 5 - Transfer OS from SD to SSD

- Wipe SSD
- Clone SD card to SSD
- Prepare SSD for boot

Commands

- ***sudo wipefs -a /dev/nvme0n1***
- ***sudo dd if=/dev/mmcblk0 of=/dev/nvme0n1 bs=4M status=progress conv=fsync***

if=/dev/mmcblk0 → **input file** = SD card (source)
of=/dev/nvme0n1 → **output file** = SSD (destination)
bs=4M → copy in 4 MB blocks (faster + safe)
status=progress → shows progress
conv=fsync → ensures data is physically written

Resize Filesystem on SSD

- ***sudo partprobe*** (reloads partition table)
- ***sudo e2fsck -f /dev/nvme0n1p2*** (checks filesystem for errors)
- ***Sudo parted /dev/nvme0n1***
- ***Print***
- ***resizepart 2 100%***

Step 5 - Transfer OS from SD to SSD

Change the boot order and reboot

Step 6 - Set Hostnames & Hosts File

- ***sudo nano /etc/hosts***

127.0.0.1 localhost

127.0.1.1 node6

172.28.8.3 master
172.28.8.4 node1
172.28.8.5 node2
172.28.8.6 node3
172.28.8.7 node4
172.28.8.8 node5
172.28.8.9 node6

Check Node connectivity

ping -c 2 master

ping -c 2 node1

Step 7 - Passwordless SSH

On Master Node

ssh-copy-id hpc@node6

Step 8 - Install MPI, GlusterFS and Slurm

- *sudo apt install -y openmpi-bin openmpi-common libopenmpi-dev*
- *sudo apt install -y glusterfs-server*
- *sudo systemctl enable --now glusterd*
- *systemctl is-active glusterd*
- *sudo apt install -y slurmd*
- *sudo apt install -y munge*

Step 9 - Setting up munge

- ***sudo scp /etc/munge/munge.key hpc@node6:/tmp/munge.key*** (copy munge key from master)
- ***sudo mv /tmp/munge.key /etc/munge/munge.key*** (Places the Munge key where the Munge service expects it on node6)
- sudo chown munge:munge /etc/munge/munge.key (Gives ownership of the key file to the Munge service user.)
- sudo chmod 400 /etc/munge/munge.key(Secures the key so only Munge can read it.)
- ***sudo systemctl restart munge*** (Restarts Munge so it loads the new authentication key.)
- ***systemctl is-active munge***

Step 10 - Setting up Slurm

Create Slurm directories

- **Creates config directory -** ***sudo mkdir -p /etc/slurm***
  Creates log directory - ***sudo mkdir -p /var/log/slurm***
- **Creates spool directory-** ***sudo mkdir -p /var/lib/slurm/slurmd***
- **Gives Slurm permission to write there -** ***sudo chown -R slurm:slurm /var/log/slurm /var/lib/slurm***
- ***scp /etc/slurm/slurm.conf hpc@node6:/tmp/slurm.conf*** (Copy conf from master)
- ***sudo mv /tmp/slurm.conf /etc/slurm/slurm.conf*** (paste it on node6)
  sudo chown root:root /etc/slurm/slurm.conf
  sudo chmod 644 /etc/slurm/slurm.conf

Step 11 - Setting up Gluster

Create Brick on node6

sudo mkdir -p /gluster/brick1
sudo chown -R root:root /gluster
sudo chmod 755 /gluster
ls -ld /gluster/brick1

Check peer status on master node

sudo gluster peer status

Detach the old version on master

sudo gluster peer detach node2 force

Step 11 - Setting up Gluster

Run on both master and node6

sudo systemctl enable --now glusterd

systemctl is-active glusterd

On master remove the already existed brick of node6

sudo gluster volume remove-brick gv0 node6:/gluster/brick1 force

On master run

sudo gluster peer probe node6

sudo gluster peer status

sudo gluster volume add-brick gv0 node6:/gluster/brick1 force

sudo mount -t glusterfs master:/gv0 /mnt/gv0

sudo systemctl daemon-reload

sudo mount -a

df -h /mnt/gv0

Step 11 - Setting up Gluster

Final unmount and mount

```
for n in node1 node2 node3 node4 node5 node6; do
  echo "====== $n ======"
  ssh -t hpc@$n "sudo umount -lf /mnt/gv0 2>/dev/null || true; sudo rm -rf /mnt/gv0;
sudo mkdir -p /mnt/gv0; systemctl is-active glusterd || sudo systemctl start glusterd;
sudo mount -t glusterfs master:/gv0 /mnt/gv0; df -h /mnt/gv0"
done
```

Step 12 - Testing code

- *sudo mkdir -p /mnt/gv0/hpc-lab/mpi*
- *sudo chown -R hpc:hpc /mnt/gv0/hpc-lab*
- *cd /mnt/gv0/hpc-lab/mpi*

Step 12 - Testing code

```
#!/bin/bash
#SBATCH --job-name=hi_demo
#SBATCH --partition=debug
#SBATCH --nodes=6
#SBATCH --ntasks=6
#SBATCH --ntasks-per-node=1
#SBATCH --time=00:02:00
#SBATCH --output=hi_%j.out
#SBATCH --error=hi_%j.err

srun --mpi=pmix
/mnt/gv0/hpc-lab/mpi/hi_master_to_all
```

```
#include <mpi.h>
#include <stdio.h>
#include <string.h>
int main(int argc, char **argv)
{
   MPI_Init(&argc, &argv);
   int rank, size;
   char message[20];
   MPI_Comm_rank(MPI_COMM_WORLD, &rank);
   MPI_Comm_size(MPI_COMM_WORLD, &size);
   if (rank == 0) {
      // Master process
      strcpy(message, "Hi");
      printf("Master (rank 0) sending 'Hi' to all %d processes\n", size-1);
      for (int i = 1; i < size; i++) {
         MPI_Send(message, strlen(message) + 1, MPI_CHAR, i, 0,
MPI_COMM_WORLD);
      }
   } else {
      // Worker processes
      MPI_Recv(message, 20, MPI_CHAR, 0, 0, MPI_COMM_WORLD,
MPI_STATUS_IGNORE);
      printf("Rank %d received message: %s\n", rank, message);
   }
   MPI_Finalize();
   return 0;
}
```

THANK YOU